

INTERNAL USE

**MEDIATEK**

# GPU直通Codebase差异说明-Audio



平台版本与架构差异说明

# 版本对齐：Yocto 同源，Android 平台切换

原平台与直通平台的版本信息如下；“保持/变化” 仅依据已提供的分支和版本说明。

| 层级      | 长安科技mp平台             | GPU直通平台                       | 结论                                                |
|---------|----------------------|-------------------------------|---------------------------------------------------|
| Android | alps-mp-u0.mp9       | alps-mp-v0.mp7+alps-mp-b1.mp7 | 1.Audio HAL引入更灵活的xml配置<br>2.Framework升级为AndroidB1 |
| Yocto   | spm-mp-scarthgap.mp5 | spm-mp-scarthgap.mp5          | 同一branch                                          |
| Kernel  | 原平台 Kernel 基线        | 沿用同一 Yocto branch             | 一致                                                |

依据：用户提供的版本信息

平台版本与架构差异说明

# Audio HAL的升级由接口、配置方式与映射能力组成

新平台在 alps-vf-mp-v0.mp7-mp-b1.mp7 中引入更灵活的音频配置方式。

01

接口机制

**HIDL → AIDL**

Audio HAL 的接口机制切换为 AIDL。

02

配置方式

**XML 配置**

新增auto\_audio\_config.xml描述音频硬件配置的方式。

03

映射对象

**PCM + Channel Mapping**

XML 可配置 PCM 节点与通道 mapping。

预期价值：配置更方便、更灵活，便于适配不同 PCM 节点与通道组合。

# Audio HAL灵活的XML配置说明

引入auto\_audio\_config.xml， auto\_audio\_config.xml 是车机音频配置文件，用于定制化AudioHAL中的相关功能

主要配置功能：

1. 配置AudioFlinger下行的Bus与ALSA节点的对应关系
2. 配置AudioFlinger下行的每一笔write的数据大小
3. 配置上下行中ALSA节点：
  - 必要硬件连接（AFE Interconnect）
  - 开启硬件的相关参数（pcm\_config）及处理数据的参数
4. 配置ADSP的混音通路
5. 配置HAL中SW Mixer的混音通路

# auto\_audio\_config.xml

# auto\_audio\_config.xml

- Introduction of auto\_audio\_config.xml
- Introduction of Lables
  - <channel\_map>
  - <bus\_stream\_maps>
  - <sw\_mixer\_routes>
  - <hal\_sw\_mixer\_routes>

# Introduction of Lables

## <channel\_map>

<channel\_map> 是为 AudioHal 中相关通路提供通道映射配置

完整配置:

```
<channel_map>  
  <chmap id="MIC_Record">  
    <map channels="2" ch_map="0x1,0x2"/>  
    <map channels="4" ch_map="0x1,0x2,0x4,0x8"/>  
  </chmap>  
</channel_map>
```

AudioHAL中能够通过id和channels获取ch\_map, 如此则可再不修改code的情况下, 通过修改xml实现不同的映射关系。

# Introduction of Lables

## <channel\_map>

客户如何在AudioHAL中增加自定义的ch\_map?

1. 在xml中增加新的chmap标签, 设定唯一的id值, 然后对不同的channels数设置ch\_map;
2. 在 **AudioConfigManager.h** 中增加对应的id值定义, 用于code中应用:

```
92  /* add new channel map id here */  
93  static const String8 AUDIO_CHMAP_MIC_RECORD("MIC_Record");  
94  static const String8 AUDIO_CHMAP_ASR_RECORD("ASR_Record");  
95  _____
```

3. AudioHAL中应用, 则调用**AudioConfigManager::getChannelMapConfig**函数即可。

```
struct aud_ch_map_cfg_t *getChannelMapConfig(const String8 Id, uint32_t channels);
```

# Introduction of Lables

## <bus\_stream\_maps>

<bus\_stream\_maps> 是为 AudioHal 中管理上下行通路的地方，用于bus和alsa节点的对接，每一个通路以子标签<bus\_stream>作为一个配置单元。

举例完整配置：

```
<bus_stream bus_num="1" hal_buffer_frames="256">
  .....<pcm_profile id="DL1" pcm_stream="DL1" rate="48000" channels="2" format="S32_LE" period_frames="256" period_cnt="4" start_threshold="0">
  .....<data_profile rate="48000" channels="2" format="S32_LE"/>
  .....<device_path>
  .....<path name="DL1_TO_I2SOUT1"/>
  .....</device_path>
  .....</pcm_profile>
</bus_stream>
```

- bus1，对应到ALSA节点的DL1；
- DL1 参数 pcm\_config取自于 pcm\_profile 中的配置；
- 硬件在SOC内部连接是DL1\_TO\_I2SOUT1；
- 数据处理读取的是data\_profile的配置。

从此配置可知，bus1 数据最终通过 I2SOUT1 输出到SOC外部。

# Introduction of Lables

## <bus\_stream>

在 xml 的 **bus\_stream** 标签中，有 属性 和 子标签

➤ 属性 **hal\_buffer\_size/hal\_buffer\_time**

◆ AF 会读取该值，从而控制 AudioFlinger Mixer 的 buffer\_size（每一笔混音大小）

➤ 子标签 **pcm\_profile**

◆ 标识属性 **id/pcm\_stream**

◆ 配置属性 **period\_frames/period\_time\_ms/period\_cnt/rate/format/channels/start\_threshold** 等属性，用以配置ALSA Driver的PCM节点；

其中，**pcm\_profile** 可以有多个，在code中根据 **id** 属性可以获取不同的**pcm\_profile**。

# Introduction of Lables

## <bus\_stream>

### 属性1: **hal\_buffer\_frames**

**单位:** 帧数, 整型值

**作用:** 在AF开机open HAL的streamout/streamin时会传入 Samplerate 与 Format 等参数, 在 AF get buffer\_size 时即以 **hal\_buffer\_frames** 计算得出返回值

**使用示例:**

```
<bus_stream.bus_num="0">
...<pcm_profile.id="DL".pcm_stream="DSP_Playback_Playback0".hal_buffer_frames="1024"/>
</bus_stream>
```

### 属性2: **hal\_buffer\_time\_ms**

**单位:** 毫秒(ms), 浮点值, 如有小数, 建议保留 2 位小数, 如 5.33/10.67 等

**作用:** 同 **hal\_buffer\_size** 类同

**限制条件:** 当 **hal\_buffer\_size** 没有在xml中设置时, 才会使用此属性

**使用示例:**

```
<bus_stream.bus_num="0">
...<pcm_profile.id="DL".pcm_stream="DSP_Playback_Playback0".hal_buffer_time_ms="21.33"/>
</bus_stream>
```

### 默认值:

当这2个属性均没有设定时, 则默认设定 **hal\_buffer\_frames** 为 **1024** 帧。

**应用优先级:** **hal\_buffer\_frames** > **hal\_buffer\_time\_ms** > **default**

# Introduction of Lables

## <pcm\_profile>

属性成员:

*id/pcm\_stream*

*period\_frames/period\_time\_ms/period\_cnt*

*rate/format/channels/start\_threshold*

限制条件: **pcm\_profile** 该必须包含 **id**, 此值是Code获取该设定的唯一标识, 如未设定, 则此 **pcm\_profile** 无效。

调用方法及其原理:

使用如下接口可在bus中找到对应的 **pcm\_profile**:

```
sp<PcmProfile> BusStreamConfig::getPcmProfileById(const char *id)
```

应用xml中 **pcm\_profile** 设定的相关参数:

```
status_t PcmProfile::applyPcmProfile(pcm_config *config, bool isCapture)
```

注意, 该函数用于覆盖 config 的初始设定, 所以config 需要传入当前的**rate/format/channels**, 因为这三个变量如不在xml中配置, 则使用当前值。

其中, **period\_frames** 与 **period\_cnt** 不管xml中是否设定, 都会有默认值覆盖掉原先的 config 中的值。

# Introduction of Lables

## <pcm\_profile>

xml 配置举例:

```
<!-- use frames in pcm_profile -->
<bus_stream bus_num="1" hal_buffer_frames="256">
  <pcm_profile id="DL" pcm_stream="DL1" rate="48000" channels="2" format="S32_LE" period_frames="256" period_cnt="4">
    <data_profile rate="48000" channels="2" format="S32_LE"/>
    <device_path>
      <path name="DL1_TO_I2SOUT1"/>
    </device_path>
  </pcm_profile>
</bus_stream>

<!-- use time_ms in pcm_profile -->
<bus_stream bus_num="1" hal_buffer_frames="256">
  <pcm_profile id="DL" pcm_stream="DL1" rate="48000" channels="2" format="S32_LE" period_times_ms="5.33" period_cnt="4">
    <data_profile rate="48000" channels="2" format="S32_LE"/>
    <device_path>
      <path name="DL1_TO_I2SOUT1"/>
    </device_path>
  </pcm_profile>
</bus_stream>
```

# Introduction of Lables

## <pcm\_profile>

### 属性1: **period\_frames**

单位: 帧数, 整型值

作用: 应用于pcm\_stream节点的 pcm\_config 参数, 如xml无设置则使用default

### 属性2: **period\_time\_ms**

单位: 毫秒(ms), 浮点值, 如有小数, 建议保留 2 位小数, 如 5.33/10.67 等

作用: 同 **period\_frames**类同

限制条件: 当 **period\_frames** 没有在xml中设置时, 才会使用此属性

### **period\_frames** 默认值:

当这2个属性均没有设定时, 则默认设定 **period\_frames = hal\_buffer\_frames**

应用优先级: **period\_frames** > **period\_time\_ms** > **default**

### 属性3: **period\_cnt**

单位: 整型值

默认值: 当 **period\_cnt** 没有在xml中设置时, **DSP 通路默认为2, AFE通路默认为3**

# Introduction of Lables

## <pcm\_profile>

属性 **rate/format/channels/start\_threshold** 都是仅当xml中设定时，才会应用。

属性归类——整型值: **rate/channels/start\_threshold**

含义: 采样率/声道数/起播帧数

属性归类——字符串: **format**

含义: 音频采样格式

可选字符串: “S16\_LE”、“S32\_LE”、“S8”、“S24\_LE”、“S24\_3LE”

# Introduction of Lables

## <data\_profile>

所属标签: **pcm\_profile**

属性成员: **rate/format/channels** (同 **pcm\_profile** 中的属性定义)

调用方法及其原理:

```
status_t applyPcmDataProfile(uint32_t *rate, uint32_t *channels, audio_format_t *format);
```

使用该成员函数即可应用xml中 **data\_profile** 设定的相关参数。

注意, 该函数用于覆盖参数中的 **rate/format/channels**, 如不在xml中配置, 则使用传入的当前值。

当前并非所有的通路都支持 **data\_profile** 的设定, 这需要SW Code中自行调用才行。

当前支持的通路如下, 其余请自行查看 Code:

1. BusNormal
2. BusDsp
3. AfeMch0
4. AfeMch1
5. AfeMch2
6. DspMch
7. HAL\_Capture\_Normal 的配置
  - 基于data provider base的类, 调用了 initPcmConfig的, 但需要根据具体情况而定)

# data\_profile 与 pcm\_profile的应用

前提：相同时间内，**pcm\_profile** 与 **data\_profile** 消耗的数据量一致。

应用目的：弥补硬件声道数不足的问题

应用一：通过将**32Bit**降为**16Bit**来提升声道数

优点：采样率不变，不需要 Codec（ADC/DAC）或外置 ADSP 做特殊的适配修改

缺点：只能传16Bit/Channel，精度会有损失

xml 配置举例：I2S6 (**48K\*8ch\*32bit**) -> AFE CM -> HAL (**48K\*16ch\*16bit**)

每帧数据都一样大：**8ch \* 32bit / 8 = 32 Bytes = 16ch \* 16bit / 8**

```
<!-- HAL CM: BE(I2S/...) -> AFE CM (UL) -> HAL -->
<bus_stream capture_scene="HAL_Capture_Mch1">
  <pcm_profile id="UL" pcm_stream="CM1" rate="48000" channels="8" format="S32_LE" period_time_ms="20" period_cnt="3">
    <data_profile rate="48000" channels="16" format="S16_LE"/>
    <device_path>
      <path name="I2SIN6_TO_CM1_8CH"/>
    </device_path>
  </pcm_profile>
</bus_stream>
```

# data\_profile 与 pcm\_profile的应用

应用二：通过提升采样率来提升声道数

优点：采样精度不变，没有精度损失

缺点：需要外置 ADSP 做特殊的适配修改，Codec（ADC/DAC）大概率不能

xml 配置举例：I2S6 (**96K\*8ch\*32bit**) -> AFE CM -> HAL (**48K\*16ch\*32bit**)

相同时间数据都一样大 (Ns)：

$$96K * Ns * 8ch * 32bit / 8 = 48K * Ns * 16ch * 32bit / 8$$

```
<!-- HAL CM: BE(I2S/...) -> AFE CM (UL) -> HAL -->
<bus_stream capture_scene="HAL_Capture_Mch1">
  <pcm_profile id="UL" pcm_stream="CM1" rate="96000" channels="8" format="S32_LE" period_time_ms="20" period_cnt="3">
    <data_profile rate="48000" channels="16" format="S32_LE"/>
    <device_path>
      <path name="I2SIN6_TO_CM1_8CH"/>
    </device_path>
  </pcm_profile>
</bus_stream>
```

# Introduction of Lables

## <sw\_mixer\_routes>

- 该标签用于配置SOC内置ADSP混音通路。
- 子标签 **sw\_mixer\_target** 作为每一个Mixer的通路配置
  - 属性 **name** 标识 mixer target 的名称
  - 属性 **ch\_in/ch\_out/ch\_map/aurisys\_on** 是标识 target 使用aurisys 框架时使用
  - 属性 **ch\_in** 也是mixer 输出的声道数
  - 属性 **ch\_map** 是**ch\_in** 与 **ch\_out** 的声道映射关系
  - 子标签 **sw\_mixer\_source** 用于标识 mixer source
    - 属性 **name** 标识 mixer source 的名称
    - 属性 **ch\_in/ch\_out/ch\_map/aurisys\_on** 是标识 source 使用 aurisys 框架时使用
    - 属性 **ch\_out** 也是进入mixer 的声道数
    - 属性 **ch\_map** 是**ch\_in** 与 **ch\_out** 的声道映射关系

# Introduction of Lables

## <sw\_mixer\_routes>

➤ 示例配置:

```
<sw_mixer_target name="sw_mixer_music" ch_in="2" ch_out="2" aurisys_on="1">
  <sw_mixer_source name="sw_mixer_playback0">
    <profile ch_in="2" ch_out="2" aurisys_on="0" ch_map="0x1,0x2"/>
  </sw_mixer_source>

  <sw_mixer_source name="sw_mixer_FM_ADSP">
    <profile ch_in="2" ch_out="2" aurisys_on="0" ch_map="0x1,0x2"/>
  </sw_mixer_source>
</sw_mixer_target>
```

# Introduction of Lables

## <hal\_sw\_mixer\_routes>

- 该标签与 **sw\_mixer\_routes** 标签定义类似，其主要是定制HAL SW Mixer通路
- 子标签 **sw\_mixer\_target** 作为每一个Mixer的通路配置
  - 属性 **address** 标识 mixer target的名称
  - 属性 **ch\_in/ch\_out/ch\_map/aurisys\_on** 是标识 target 使用aurisys 框架时使用
  - 属性 **ch\_in** 也是mixer 输出的声道数
  - 属性 **ch\_map** 是**ch\_in** 与 **ch\_out** 的声道映射关系
  - 子标签 **sw\_mixer\_source** 用于标识 mixer source
    - 属性 **address** 属于mixer source的名称
    - 属性 **ch\_in/ch\_out/ch\_map/aurisys\_on**是标识 source 使用 aurisys 框架时使用
    - 属性 **ch\_out** 也是进入mixer 的声道数
    - 属性 **ch\_map** 是**ch\_in** 与 **ch\_out** 的声道映射关系

# Introduction of Lables

## <sw\_mixer\_routes>

### ➤ 示例配置:

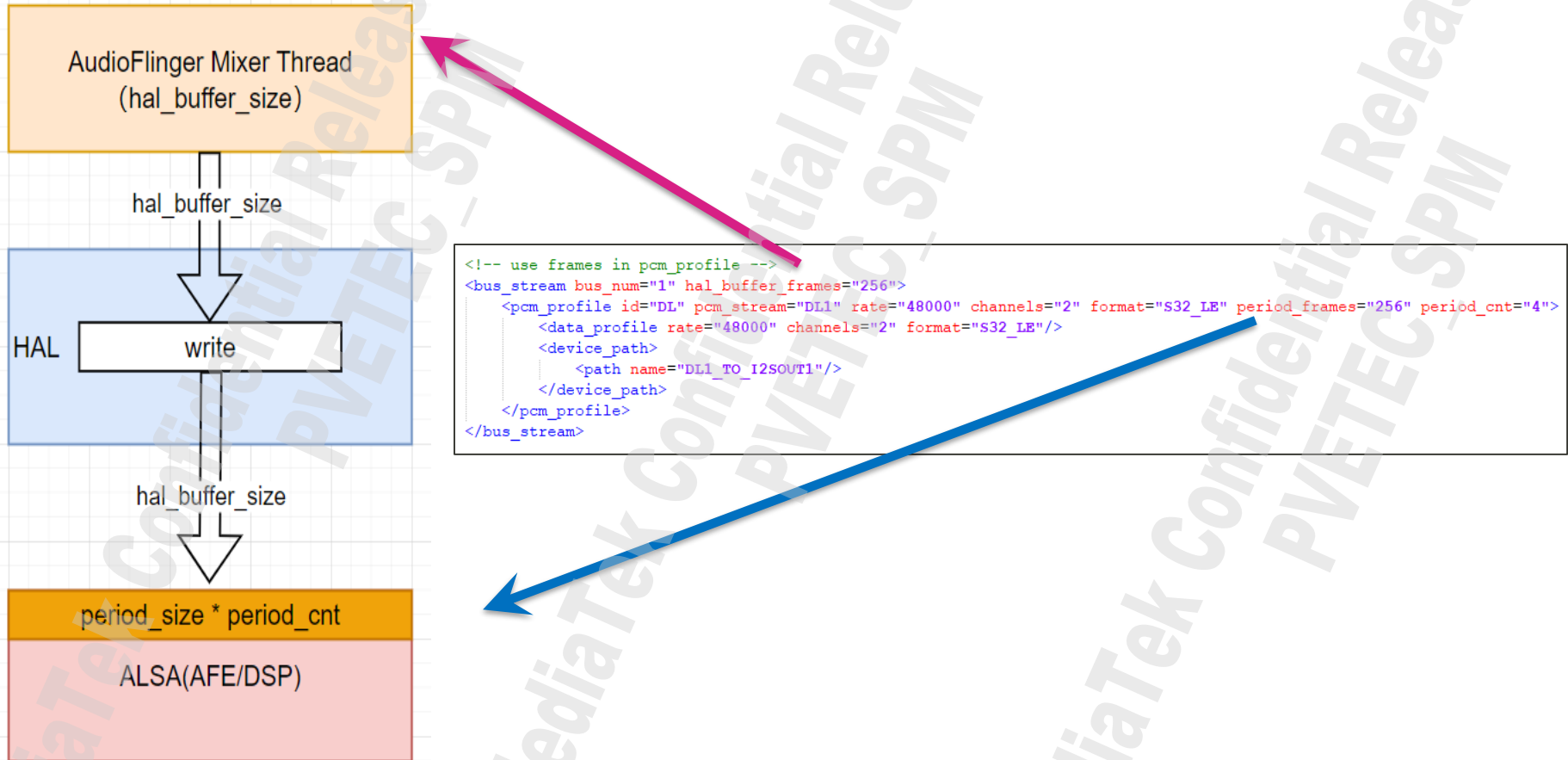
```
<!-- HAL SW Mixer Config -->
<hal_sw_mixer_routes>
  <sw_mixer_target address="bus90_internal_use" ch_in="2" ch_out="2" aurisys_on="0" type="AUDIO_DEVICE_OUT_BUS">
    <sw_mixer_source address="bus80_playback_out" >
      <profile ch_in="2" ch_out="2" aurisys_on="0" ch_map="0x1,0x2"/>
    </sw_mixer_source>

    <sw_mixer_source address="bus81_playback_out" >
      <profile ch_in="2" ch_out="2" aurisys_on="0" ch_map="0x1,0x2"/>
    </sw_mixer_source>

    <sw_mixer_source address="bus82_playback_out" >
      <profile ch_in="2" ch_out="2" aurisys_on="0" ch_map="0x1,0x2"/>
    </sw_mixer_source>
  </sw_mixer_target>
</hal_sw_mixer_routes>
```

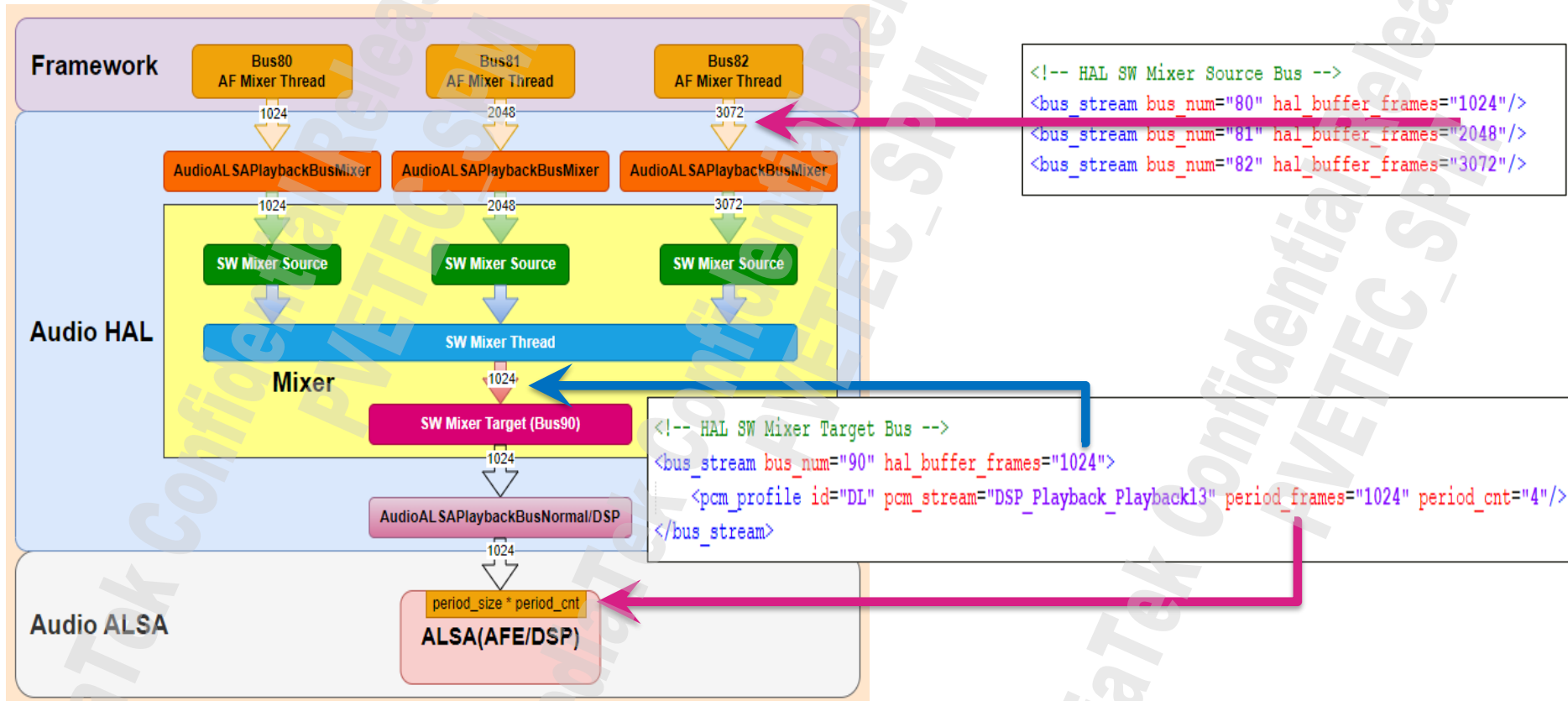
# Introduction of Lables

## <hal\_sw\_mixer\_routes>



# Introduction of Lables

## <hal\_sw\_mixer\_routes>



**MEDIATEK**

everyday genius